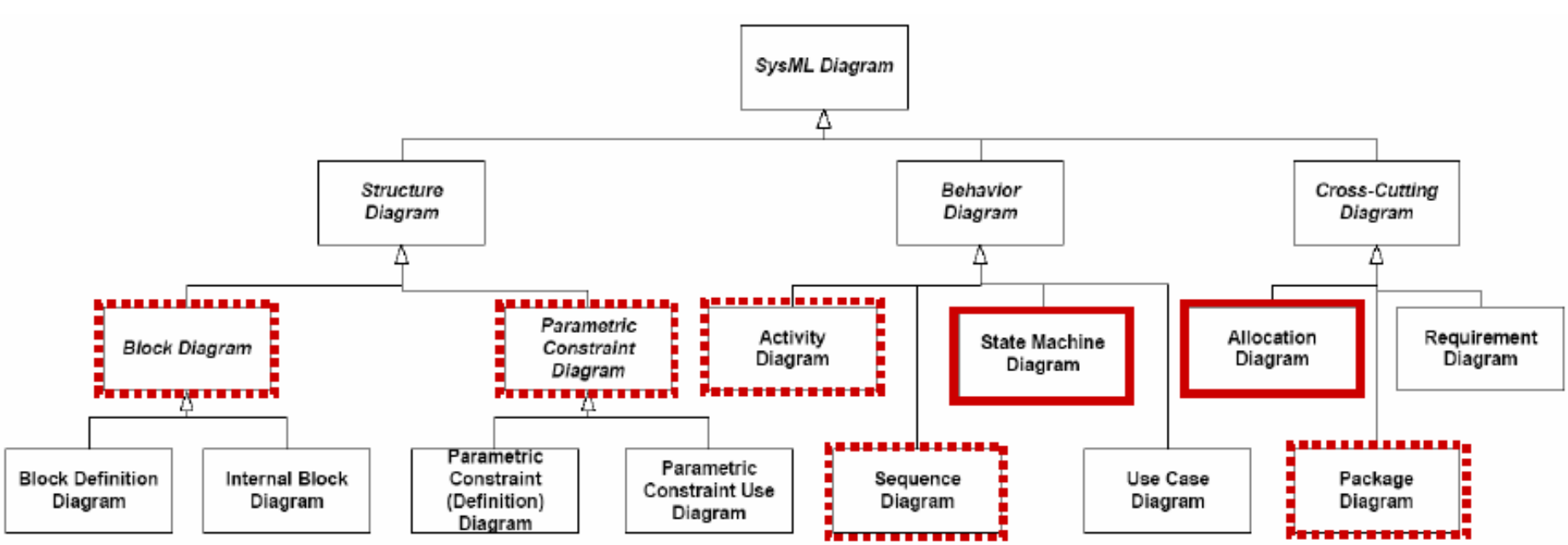
HOCHSCHULE
HAMM-LIPPSTADT

Faezeh Pasandideh E-Mail: faezeh.pasandideh@hshl.de

Stefan Henkler E-Mail: stefan.henkler@hshl.de

HOCHSCHULE HAMM-LIPPSTADT

1. Introduction
2. Methods
3. Analysis
4. Design
5. Advanced Design Concepts
6. Discussion & Summary
7. Bibliography



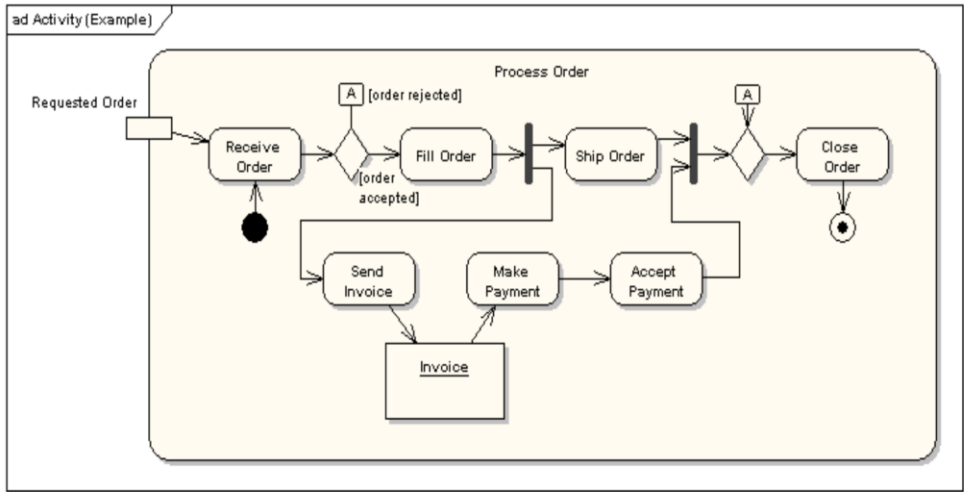
- **(5) Activity Diagram**
- (6) Sequence Diagram
- (7) Allocation Diagram

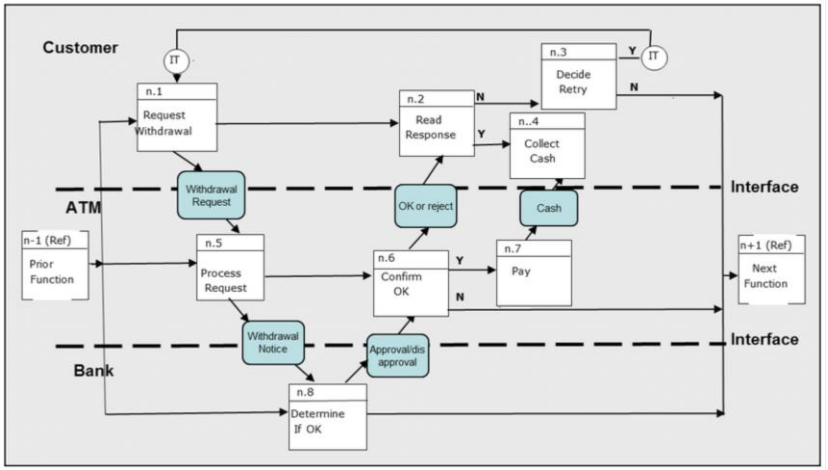
(3) Activity Diagrams

- What the system does internally (the flow of actions).
- Activities specify **sequential and concurrent** behaviors that are connected by **control flows** and **object flows**.
- Activities can be **nested** or **atomic**; in the latter case they are referred to as **actions**.
- Activity diagrams are analogous to **Extended Functional Flow Block Diagrams** (EFFBDs).

- **Application**
  - Can be used to specify the causal/functional behavior of a system.

(3) Activity Diagrams vs EFFBD





Activity Diagram

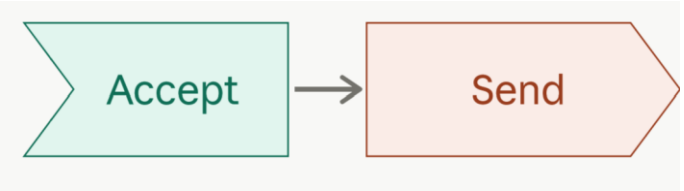
model *behavioral workflows* (software-centric)

EFFBD

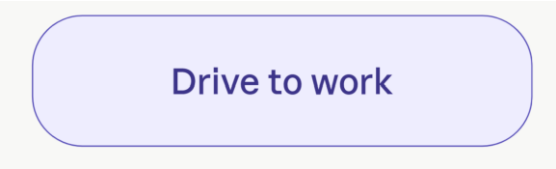
model *system functions and their logical execution order*

6 Basic Actions

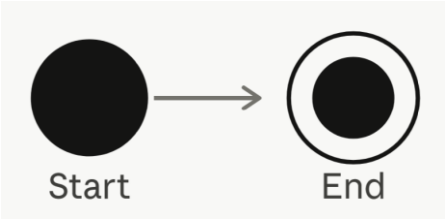
- AcceptEventAction/ SendSignalEvent



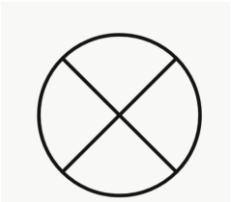
- Action



- InitialNode/ActivityFinal

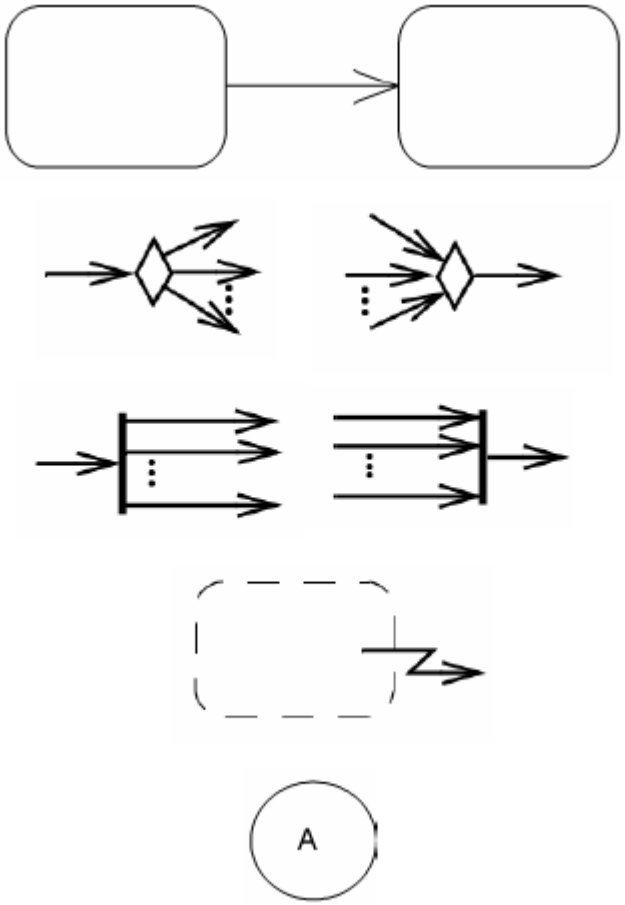


- FlowFinal



Question: The difference between ActivityFinal & FlowFinal?

7 Control Flow

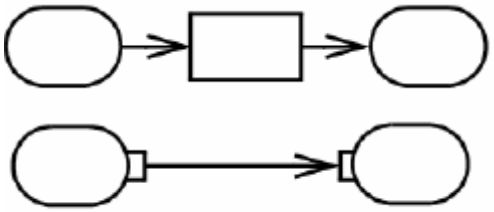


- ControlFlow
- DecisionNode/MergeNode
- ForkNode/JoinNode
- InterruptibleActivityRegion
- Page connectors: used to connect flows across multiple pages.

Question: The difference between MergeNode& JoinNode?

8 Object Flow (1/2)

Different types



- **ObjectNode:** items that are flowing through the activity and happen to be contained by the object node at the time the link exists.

9 Object Flow (1/2)

Different types



HSUV is moving and brake pedal sensor is continuedly reading pressure

«noBuffer»

- **NoBuffer:** tokens arriving at the node, which are refused by outgoing edges or refused by actions for object nodes, which are input pins, are **discarded**.

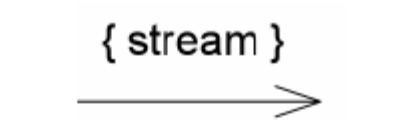
«overwrite»

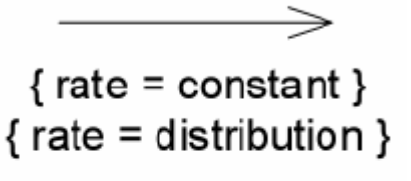
- **Overwrite:** a token arriving at a full object node **replaces the ones already there** (a full object node has as many tokens as allowed by its upper bound).

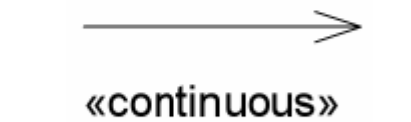
«optional»

- **Optional/Required:** the lower multiplicity must be **equal to zero**. Otherwise, the lower multiplicity must be **greater than zero**, which is called “**required**”.

10 Object Flow (2/2)



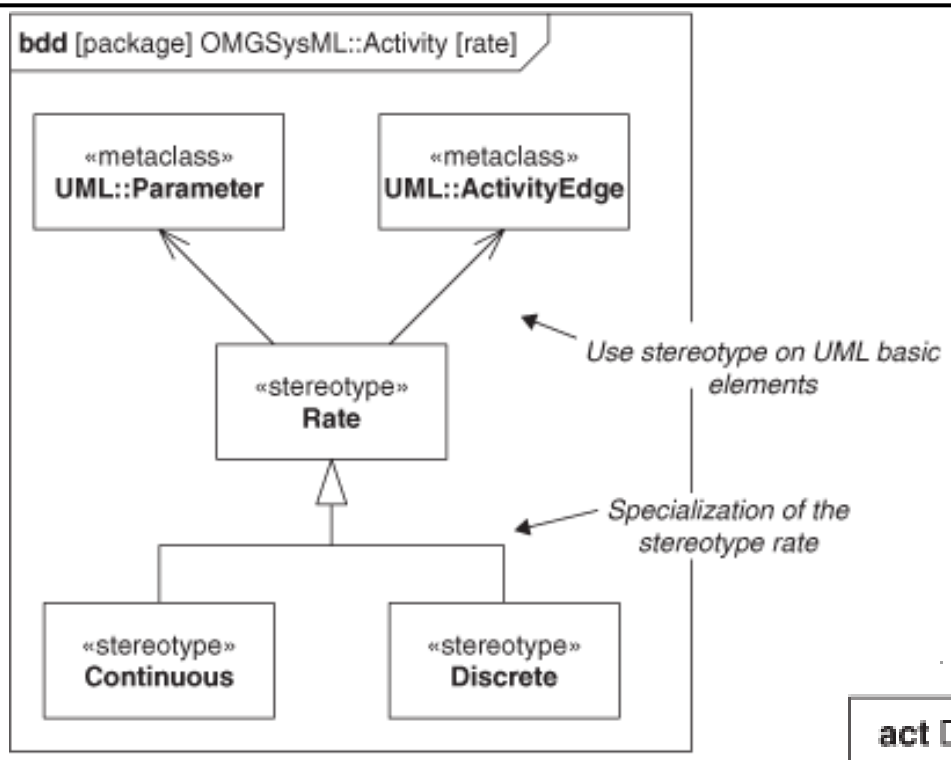


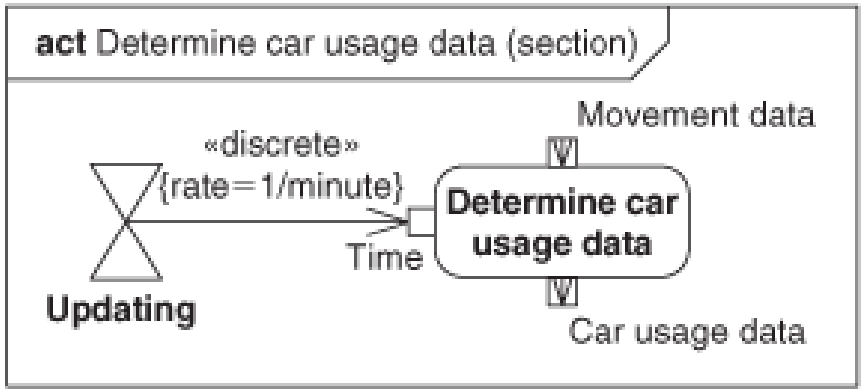


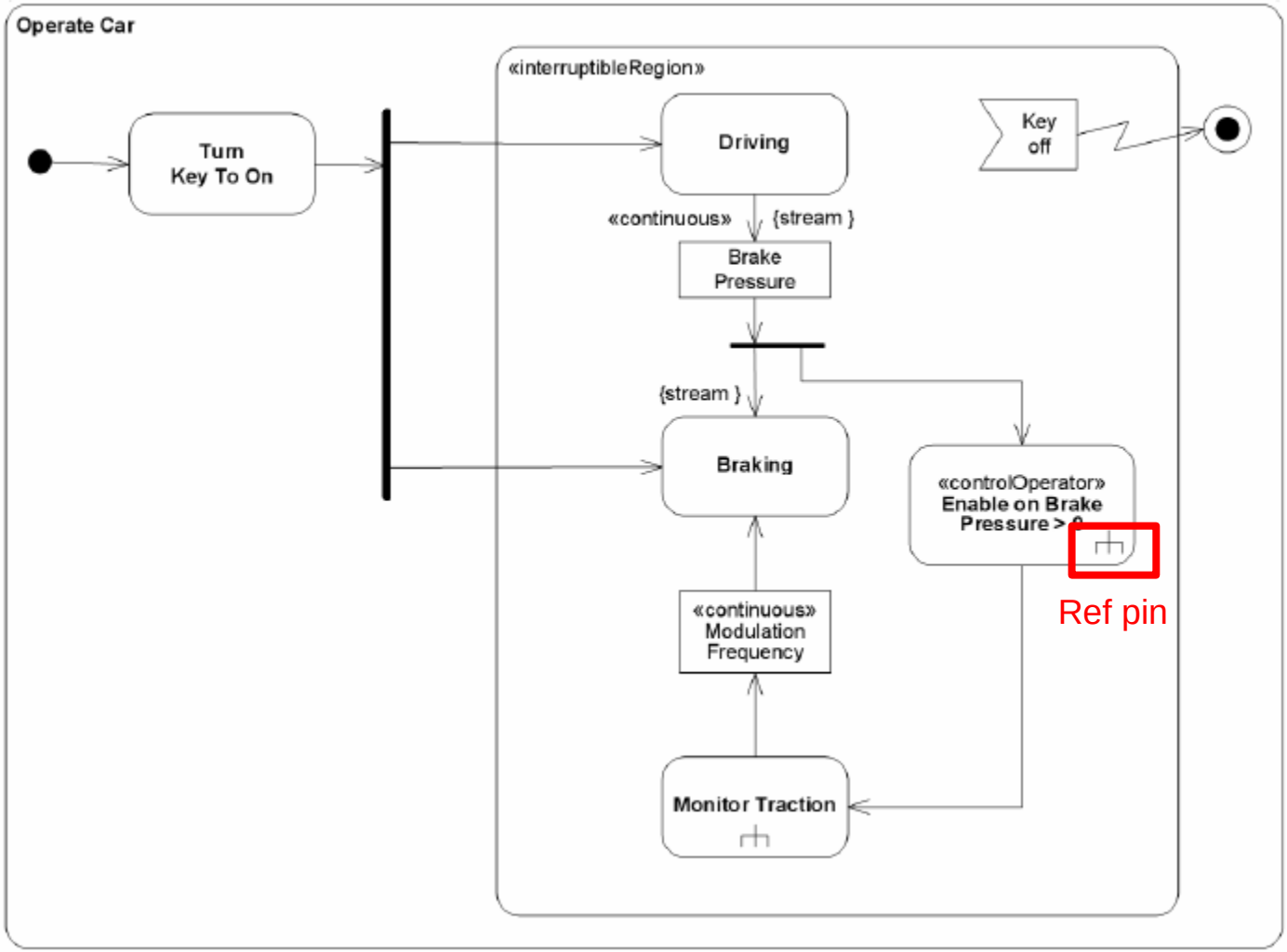


- **Stream:** supports the input and output of items **while a behavior is executing**, rather than only when the behavior starts and stops.
- **Rate:** specifies the **rate over time** that objects and values traverse the edge, that is, the rate they leave the source node and arrive at the target.
- **Continuous:** treated as infinitesimal (e.g.,water flowing a pipe) or entities sufficiently small enough to treat as continuously flowing (e.g., ball bearings in a factory).
- **Discrete:** a kind of Rate stereotype representing a rate of flow for items treated as individuals for the purpose of the application, for example, cars in a car factory.

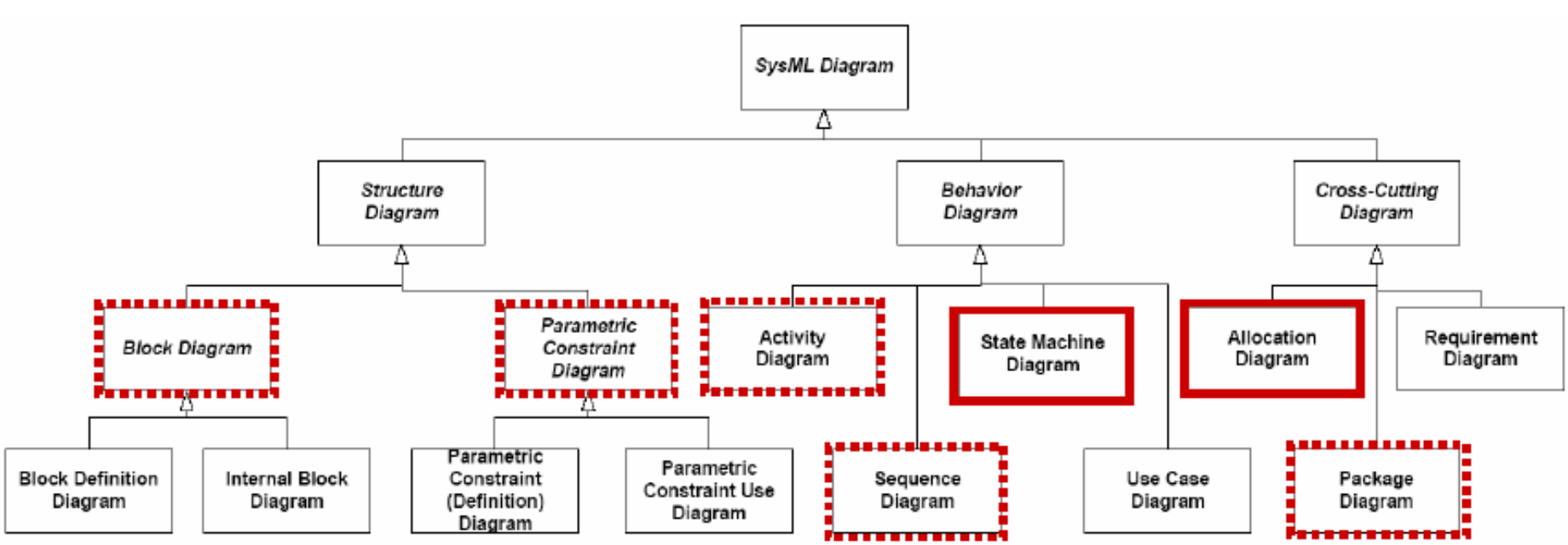
Object flow


bdd [package] OMGSysML::Activity [rate]
«metaclass»
UML::Parameter
«metaclass»
UML::ActivityEdge
«stereotype»
Rate
Use stereotype on UML basic elements
Specialization of the stereotype rate
«stereotype»
Continuous
«stereotype»
Discrete


act Determine car usage data (section)
Movement data
«discrete»
{rate=1/minute}
Time
Determine car usage data
Updating
Car usage data



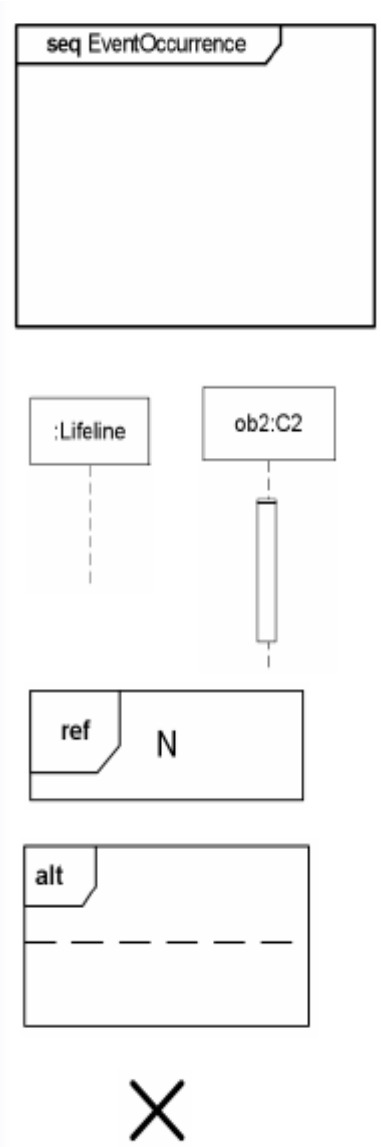
Question: What happens to the Braking thread if the driver turns the key off?



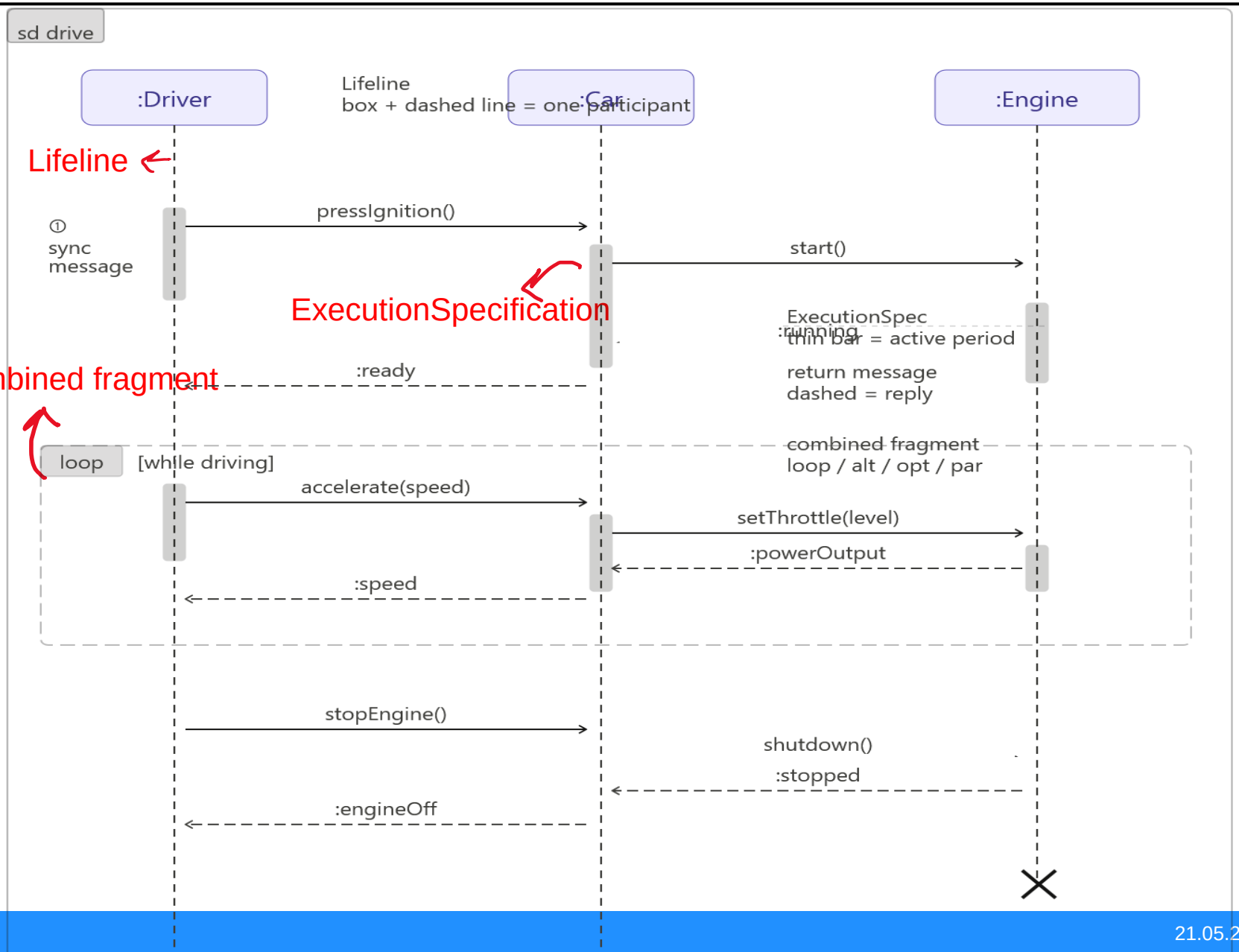
- (5) Activity Diagram
- **(6) Sequence Diagram**
- (7) Allocation Diagram

(4) Sequence Diagrams

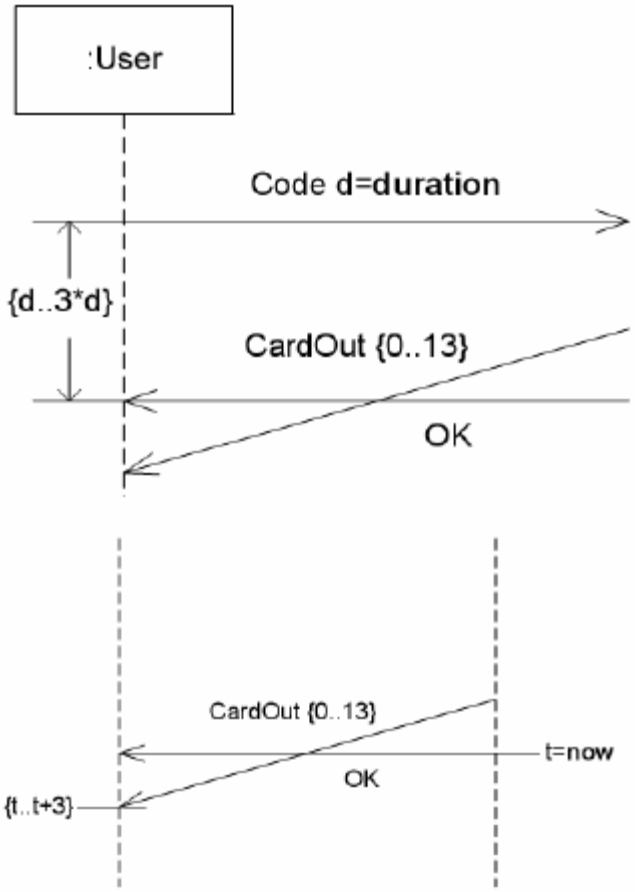
- Sequences diagrams can be used to model **communications among block structures** arranged in **time order (who talks to who and when)**.
- A Sequence diagram specifies a series of interactions in terms of message flows. A message combines **control and data-flow.**
- **Application**:
  - Can be used to specify the required interactions between the elements of a system.



- Sequence Diagram Frame
- Lifeline
- ExecutionSpecification
- InteractionUse
- CombinedFragment
- DestructionEvent


sd drive
:Driver
Lifeline
box + dashed line = one participant
:Car
:Engine
Lifeline
pressIgnition()
① sync message
start()
ExecutionSpecification
ExecutionSpec :running
thin bar = active period
:ready
return message
dashed = reply
Combined fragment
combined fragment
loop / alt / opt / par
loop
[while driving]
accelerate(speed)
setThrottle(level)
:powerOutput
:speed
stopEngine()
shutdown()
:stopped
:engineOff

Time Restrictions (UML)



A deadline: you have 3 time until to respond after recieving the card

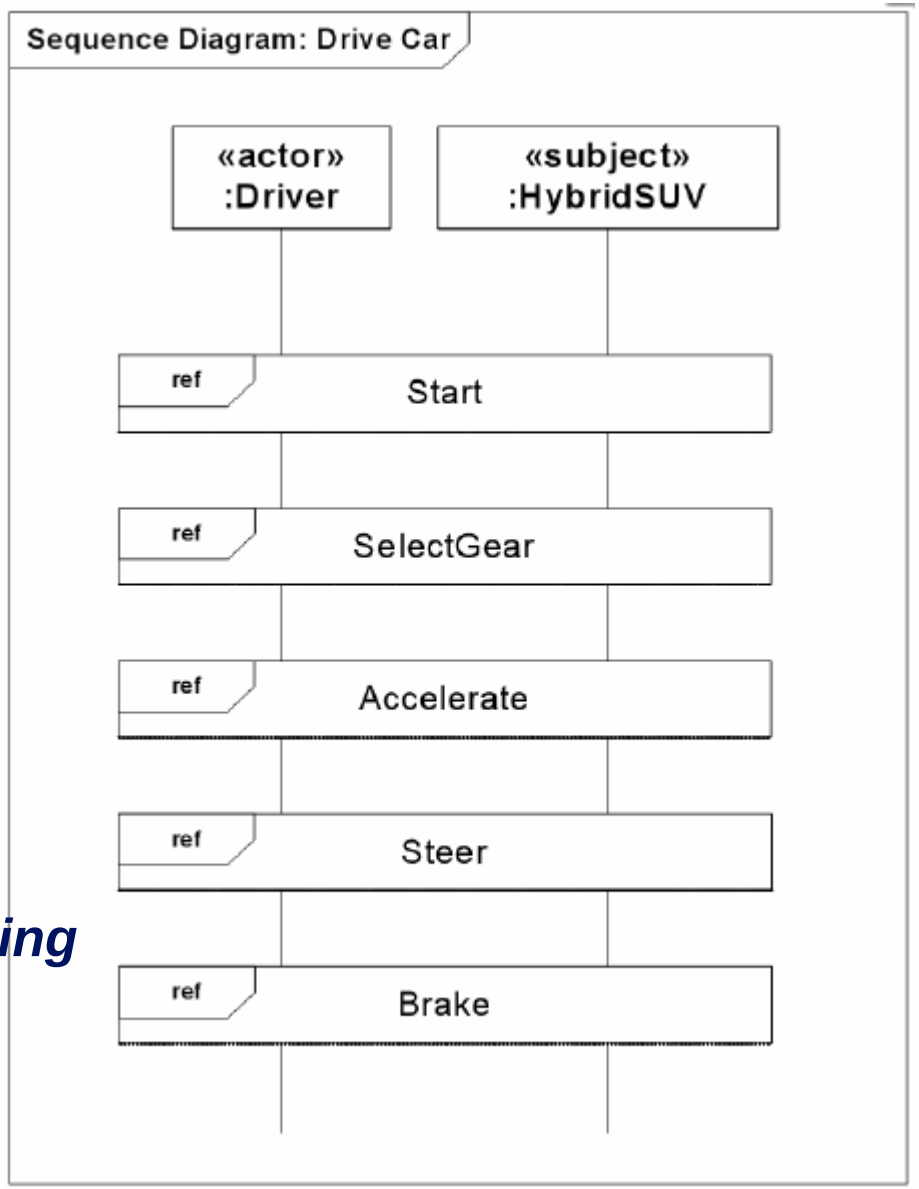
Countdown window

- Duration Constraint/ Duration Observation
  - Passage of time
  - Identifies two occurences (start and end occurrence)
- Time Constraint/ Time Observation
  - Applies to a single occurrence
  - With respect to the expression t=now

Example

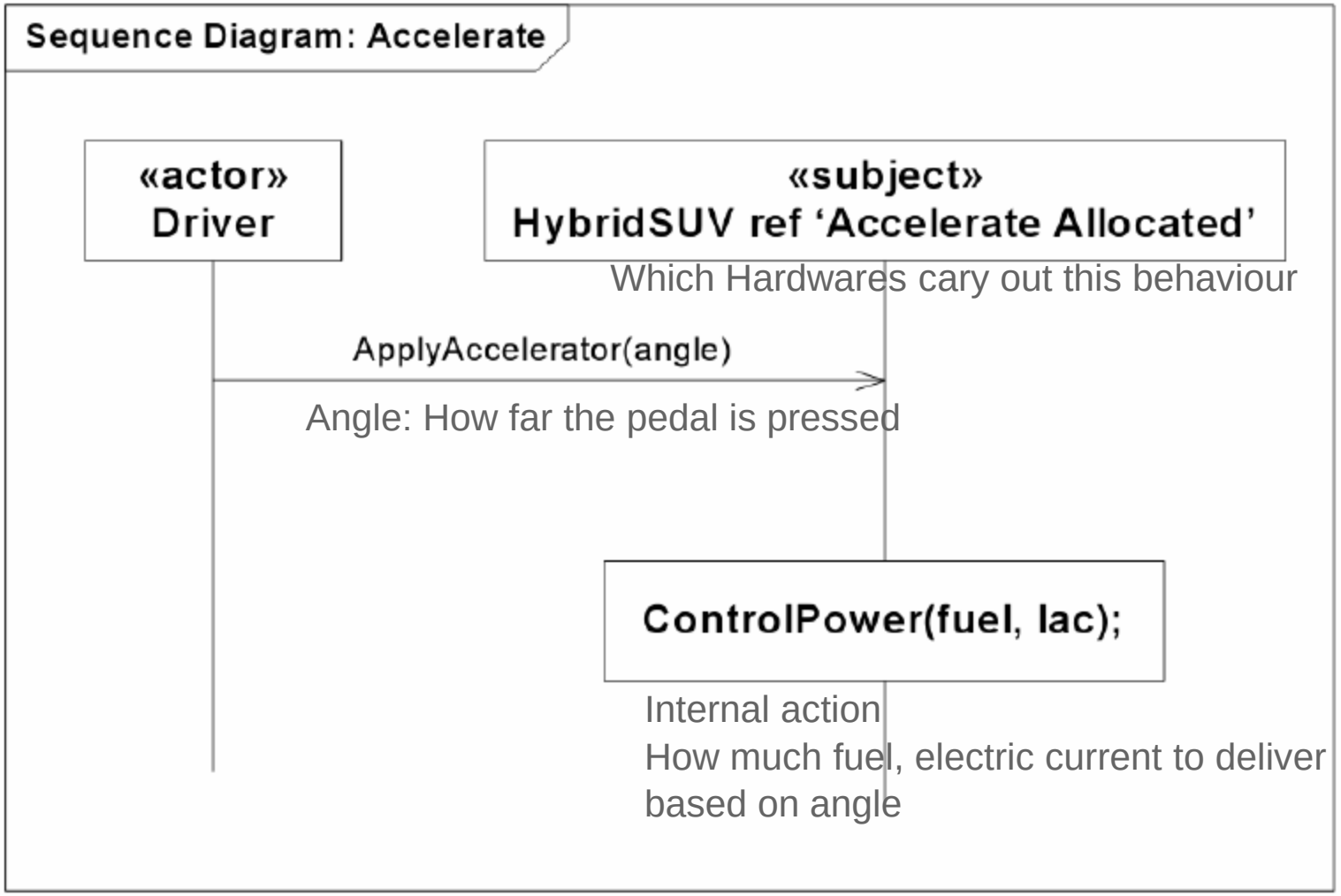
Drive Car

- High level view which references other sequence diagrams via an InteractionUse
- A big picture, overview, high level, table of content



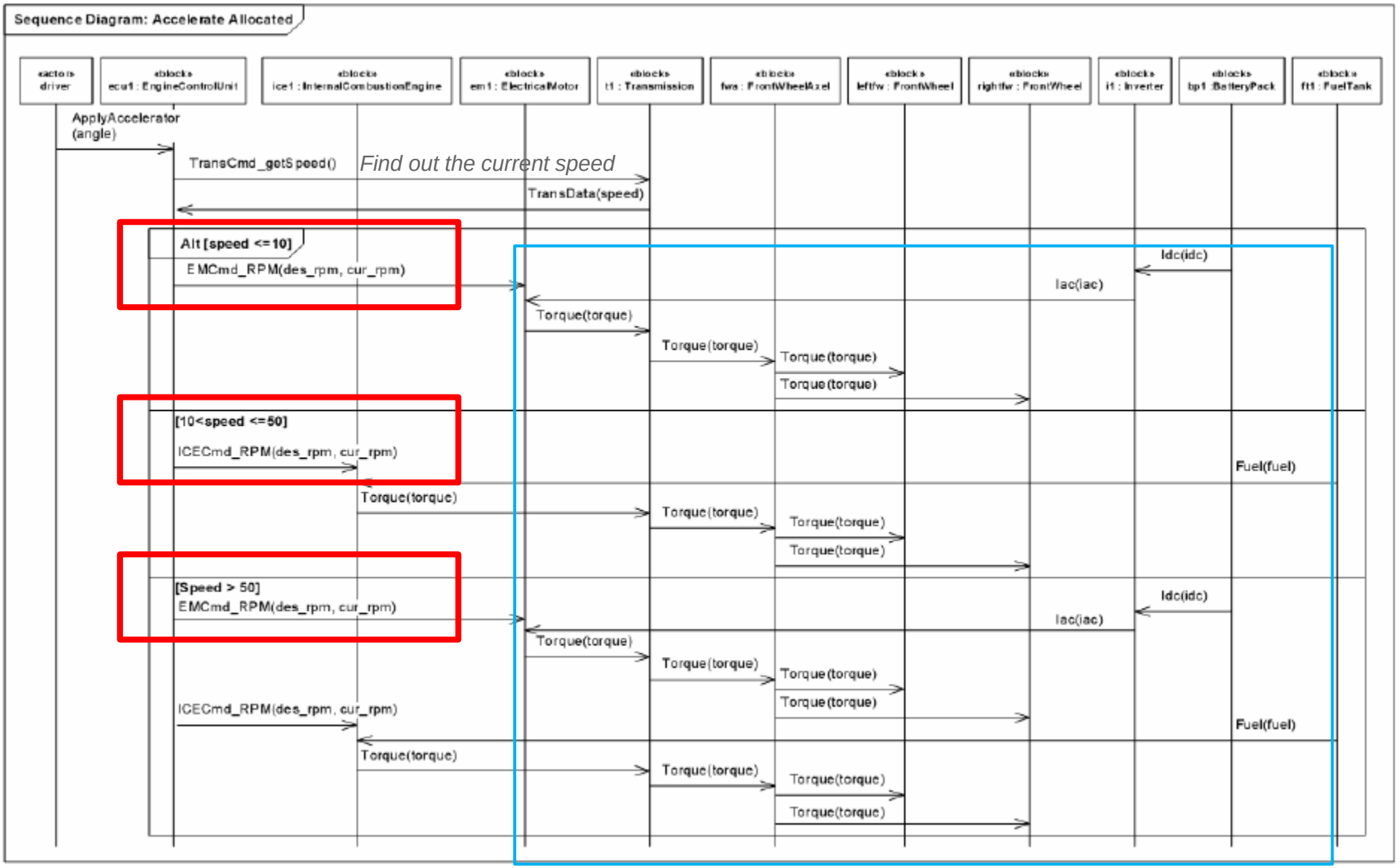
Question: Why this is better than putting all the details in one giant diagram?

Black-box View

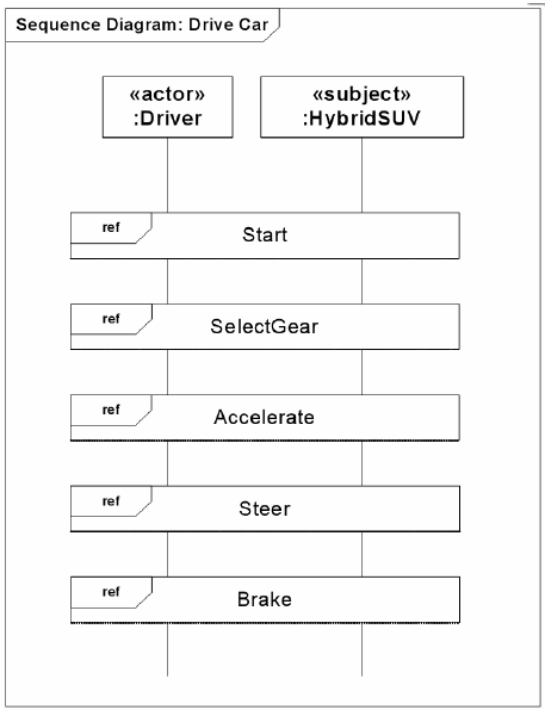


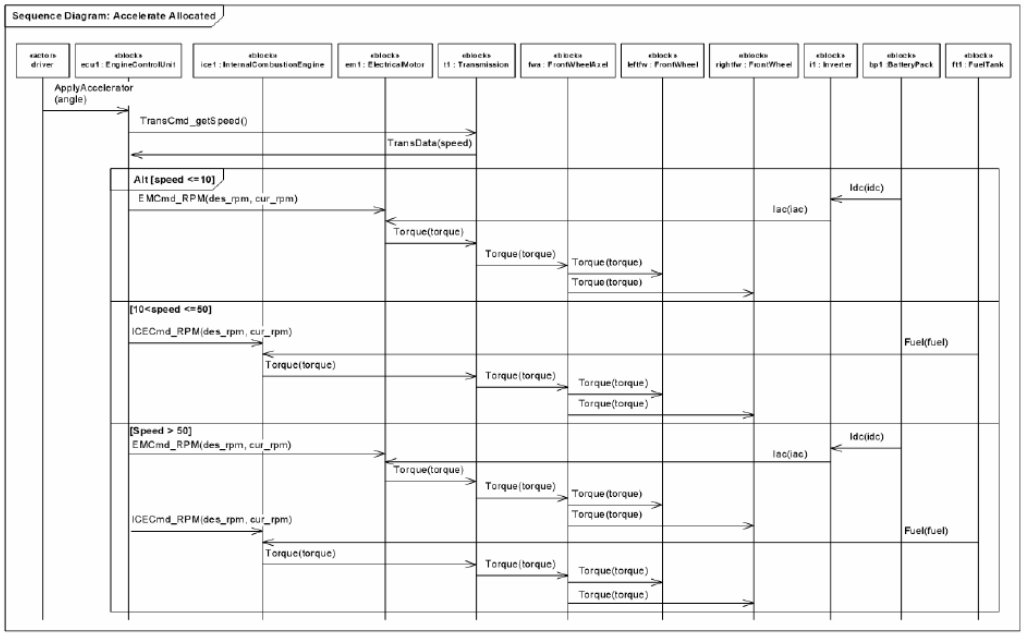
▸20 White-Box View

HOCHSCHULE



BlackBox and WhiteBox Diagrams





Question: Why do we need both diagrams, why not just the white-box?

HOCHSCHULE HAMM-LIPPSTADT

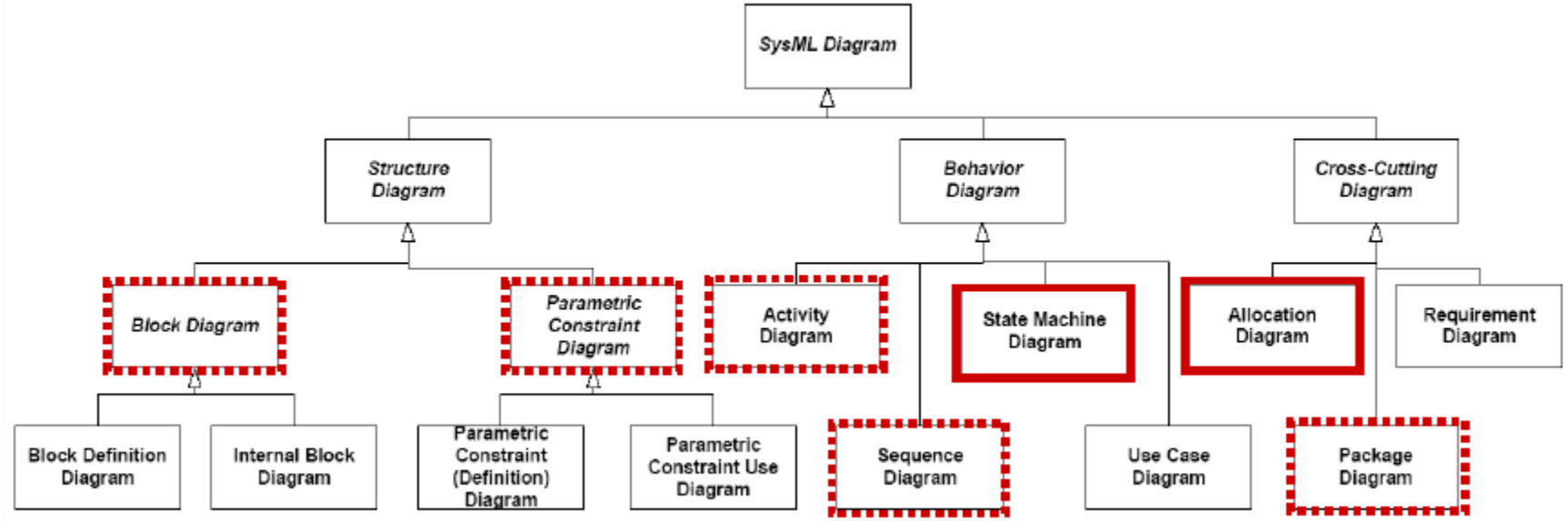
1. Introduction
2. Methods
3. Analysis
4. Design
5. Advanced Design Concepts
6. Discussion & Summary
7. Bibliography

V.4 Design

- The designer focuses on the solution of the problem (solution domain) which involves many tasks (subsystem decomposition, selection of the hardware platform, data management system, etc.).
- The design model therefore should describe the solution but may also include required or existing structure and behavior of the application identified during analysis.
- Main stake holders: designer, implementer

Analysis vs. Design

- The exact boundary between analysis and design is hard to determine. But there is a different purpose:
  - Analysis concerns the description of the **problem** and the user requirements
  - Design concerns the construction of a **solution** which satisfies the previously recorded requirements.
- There are no miracles
  - transition to design must transform analysis objects into design objects.



- **(5) State Machine Diagram**
- (6) Allocation Diagram

What can/should be Designed?

- (1) Block Diagrams
  - Construct internal structures and interfaces
- (2) Parametric Constraint Diagram
  - Define which dependencies/constraints should hold between given elements as well as invented elements
- (3) Activity Diagrams
  - Scenarios describing the flow of activities including the internal processing
- (4) Sequence Diagrams
  - Required/Likely interaction scenarios including internal elements
- (5) State Machine Diagram
  - Complete state-dependent reactive behavior of constructed elements
- (6) Allocation Diagram
  - Describe/record which elements fulfill which requirements

(5) State Machine Diagram

- The State Machine can be used to model discrete behavior through finite state transition systems in terms of its transitions and states.

- State Machine: Same event: different response

Press pedal at 50km/h>only ele Motor, at 80km/h both motors (can't be shown by Seq, ACT)

- Object does not always behave the same way

Traffic light : Red vs Green, door: opened vs closed

- At any moment the object is in exactly ONE state and certain events cause it to switch states(transition).

(5) State Machine Diagram

- The **activities** that are invoked during the **transition**, **entry**, and **exit** of the states are specified along with the associated event and guard conditions.
- **Every transition:**
- **Event**: what triggers it: buttonPressed
- **Guard**: a condition that must be True: speed>80km/h
- **Activity**: an action : presspedal()



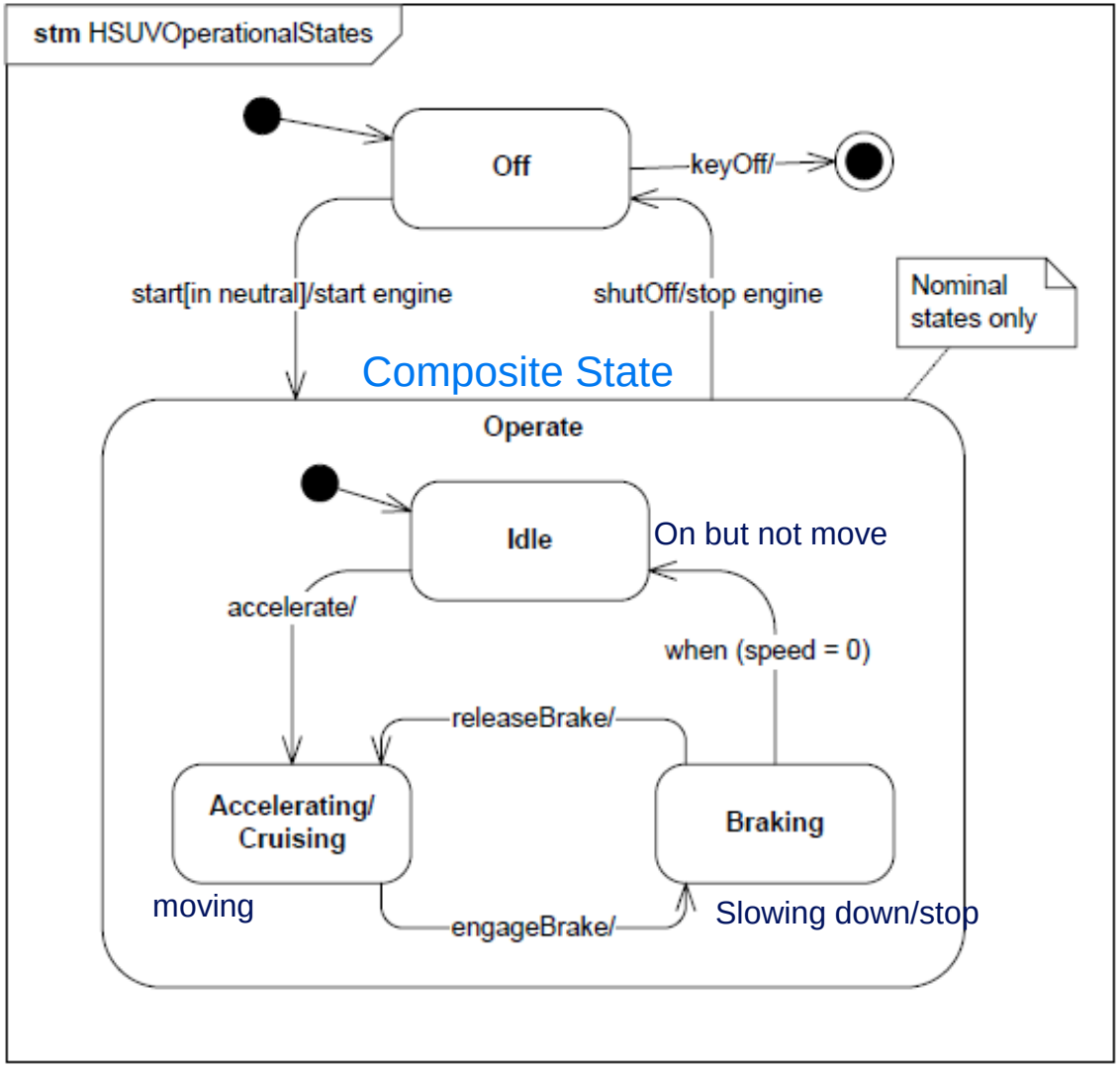
- **Activities** that are invoked while being in a state are specified as **do Activities**, and can be either **continuous** or **discrete** (Hybrid behavior!)
- A composite state has nested states that can be sequential or concurrent.

(5) State Machine Diagram

Question: in the HybridSUV ,what states do you THINK the car could be in?

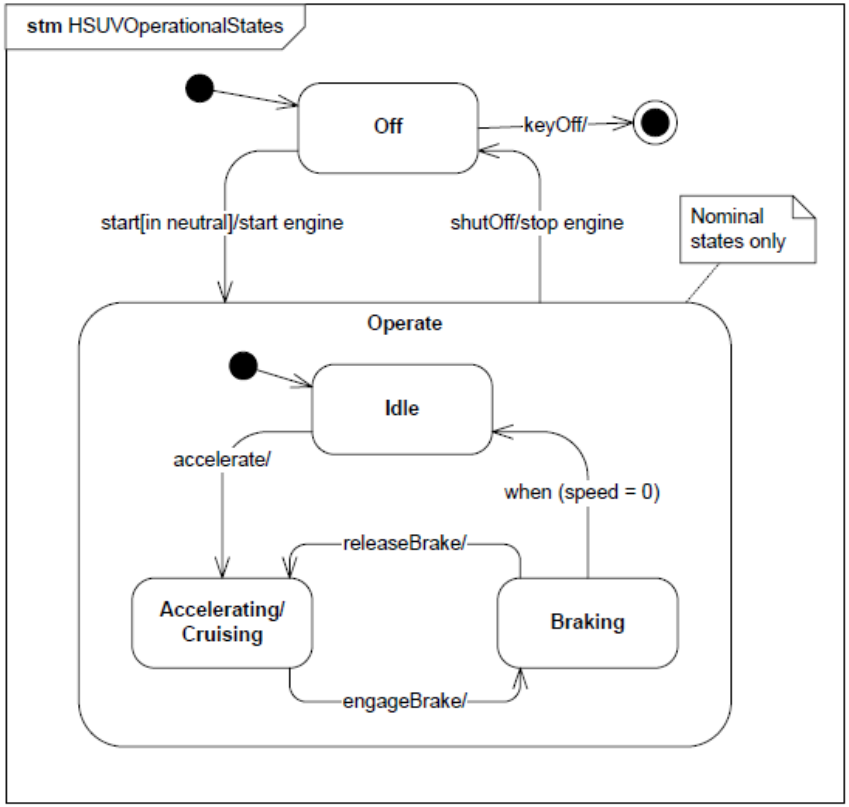
What modes does a car have?

Operational States (Drive)



Transition notation:
trigger[guard]/action

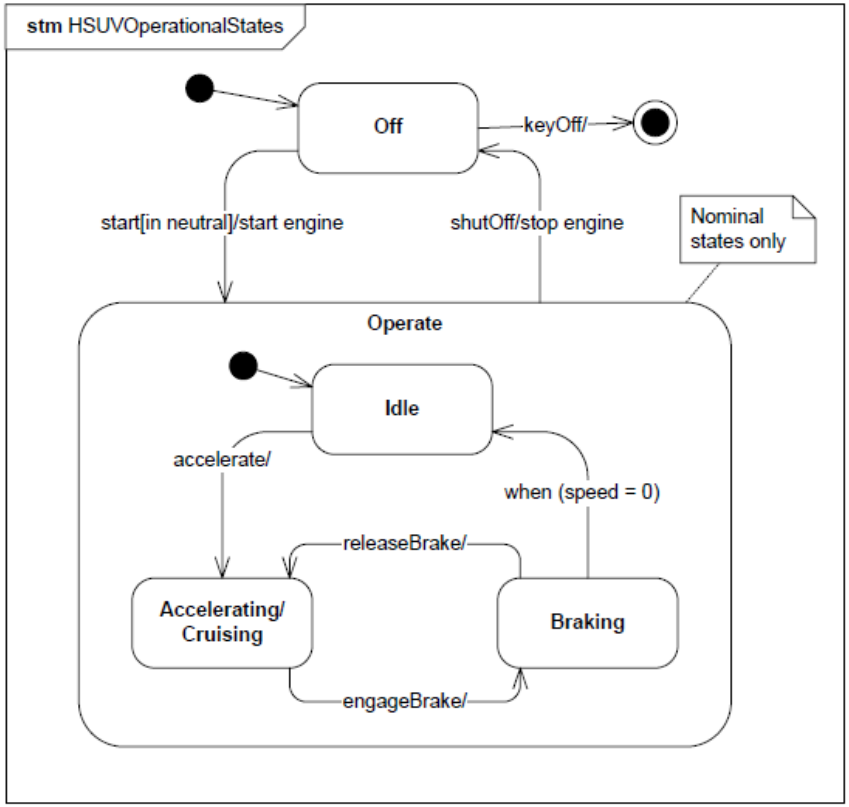
Operational States (Drive)



Transition notation:
trigger[guard]/action

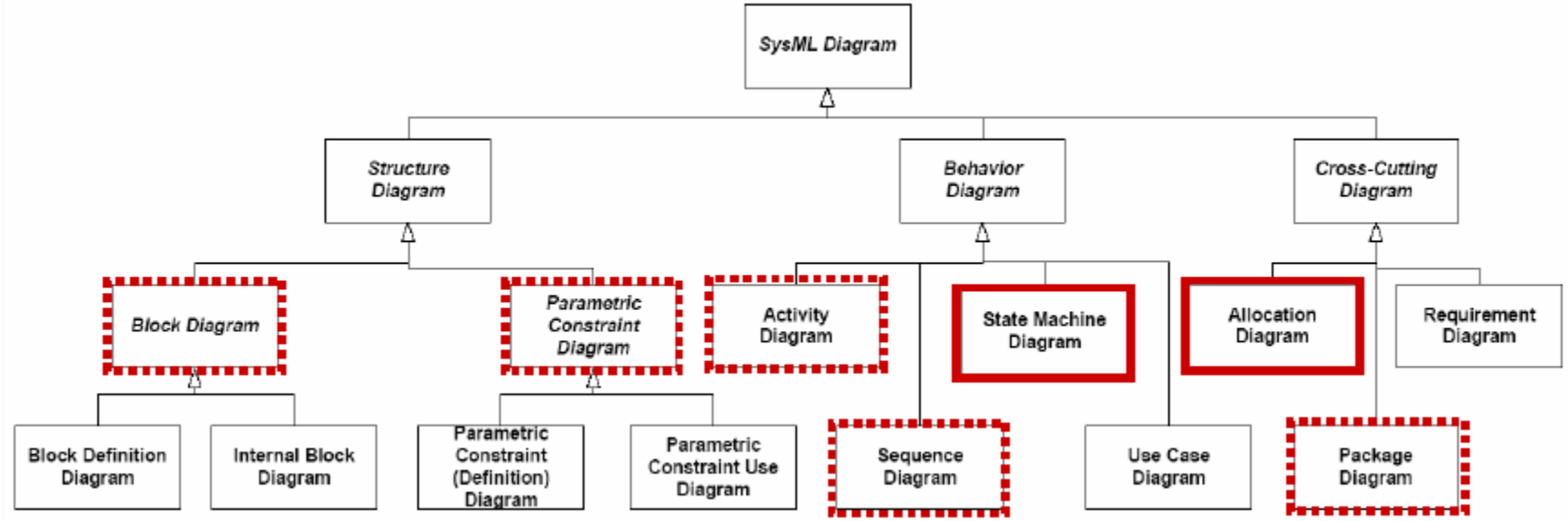
Question: What would happen if we put the ShutOff transition only on the Idle sub-state?

Operational States (Drive)



Transition notation:
trigger[guard]/action

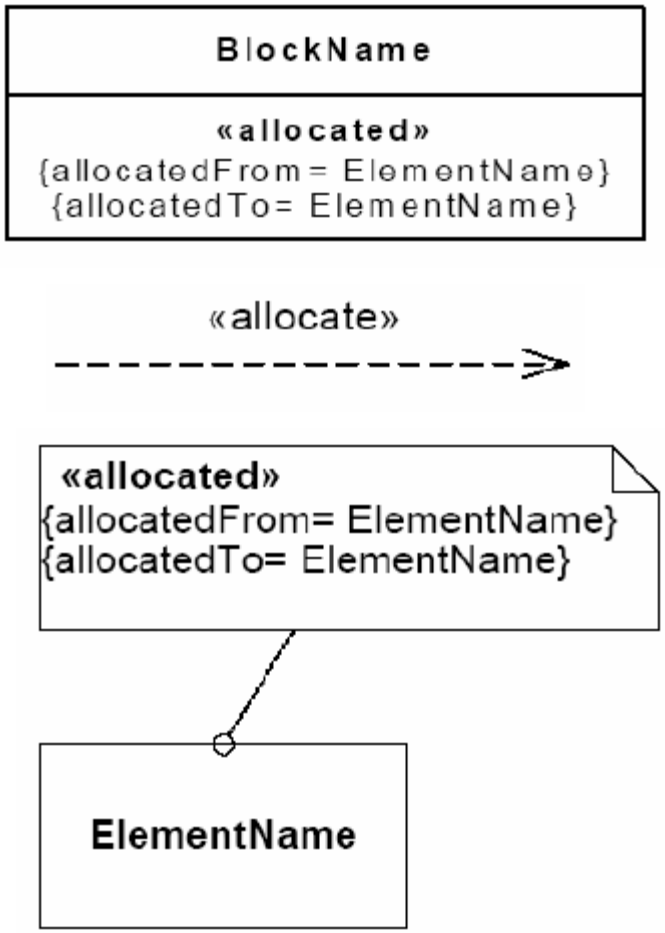
Question: Is there anyway to go from Accelerating/Cursing to Idle without going through Braking first?



- (5) State Machine Diagram
- **(6) Allocation Diagram**

Allocation Diagram

- Allocation is the term used by systems engineers to denote the **organized cross-association (mapping)** of elements within the various structures or hierarchies of a user model. *Who is responsible for what*
- Allocation is the term used by systems engineers to describe a **design decision** that **assigns responsibility for meeting a requirement** (requirements allocation) or implementing a behavior (functional allocation) to structural elements of the system.
- The allocation relationship can provide an effective means for **navigating** the model by establishing **cross relationships**, and ensuring the various parts of the model are properly integrated.



Allocation Diagram

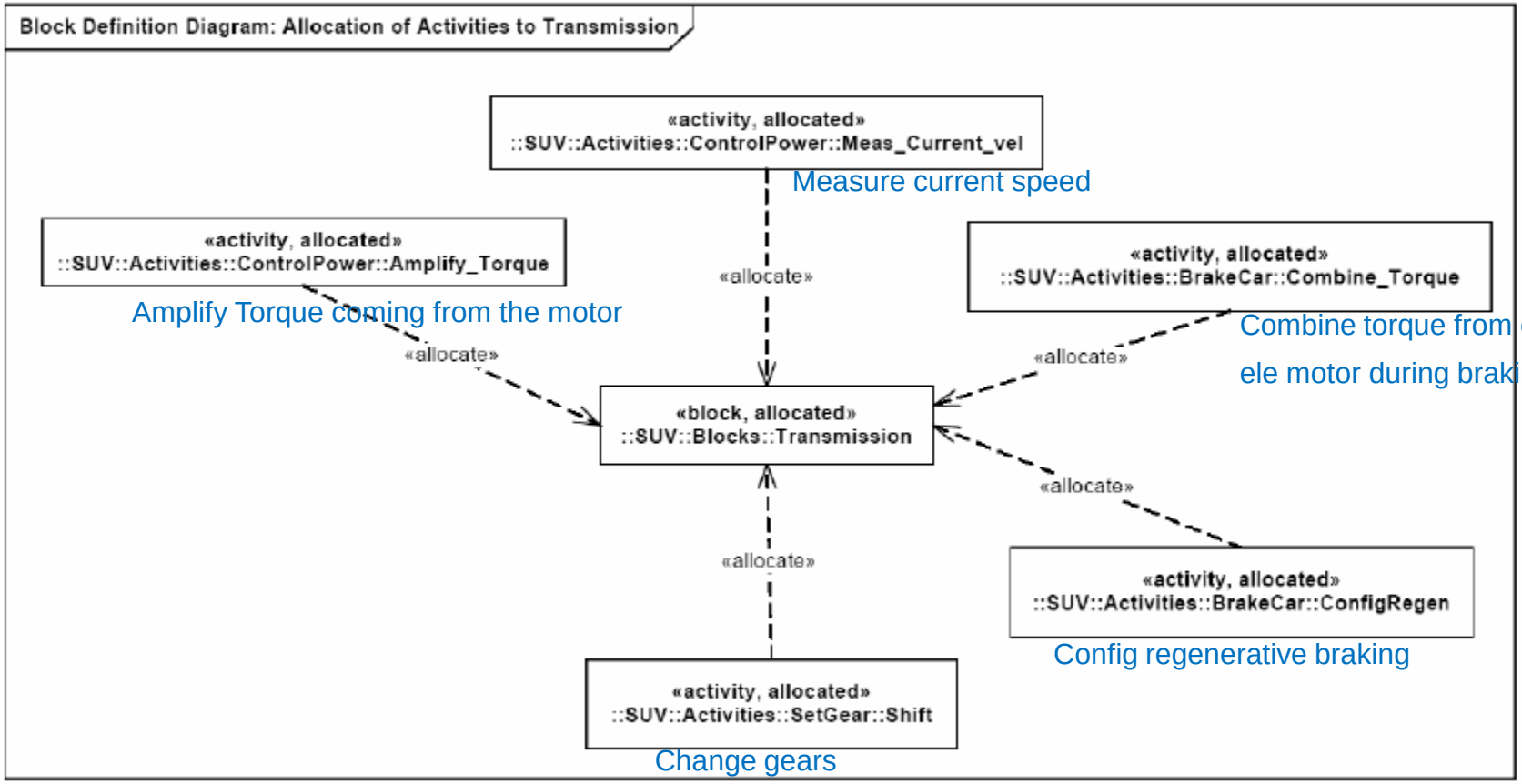
- Without allocation: model has three disconnected worlds



- With allocation



Example: Allocation Behavior



It answers: Which functions does the Transmission hardware block perform

Transmission:physical hardware: gearbox

Example: Allocation Behavior



- The top-level system
- The behavior model layer
- The parent activity (function group)
- The specific sub-activity being allocated

Table view

HOCHSCHULE HAMM-LIPPSTADT

table [package] On-board computer control [allocations]

type	name	end	relation	end	type	name
port	k:Card readerPort	from	allocateStructure	to	port	r:RS232
action	Transmit card data	from	allocateBehavior	to	block	Customer card
action	Read customer card	from	allocateBehavior	to	block	Card reader
action	Identify customer	from	allocateBehavior	to	block	On-board computer control

matrix [package] On-board computer control [allocations]

Source	Target			
	r:RS232	Customer card	Card reader	On-board computer control
k:Card readerPort	allocate			
Transmit customer data		allocate		
Read customer card			allocate	
Identify customer				allocate

Table view

table [package] On-board computer control [allocations]

type	name	end	relation	end	type	name
port	k:Card readerPort	from	allocateStructure	to	port	r:RS232
action	Transmit card data	from	allocateBehavior	to	block	Customer card
action	Read customer card	from	allocateBehavior	to	block	Card reader
action	Identify customer	from	allocateBehavior	to	block	On-board computer control

matrix [package] On-board computer control [allocations]

Source	Target			
	r:RS232	Customer card	Card reader	On-board computer control
k:Card readerPort	allocate			
Transmit customer data		allocate		
Read customer card			allocate	
Identify customer				allocate

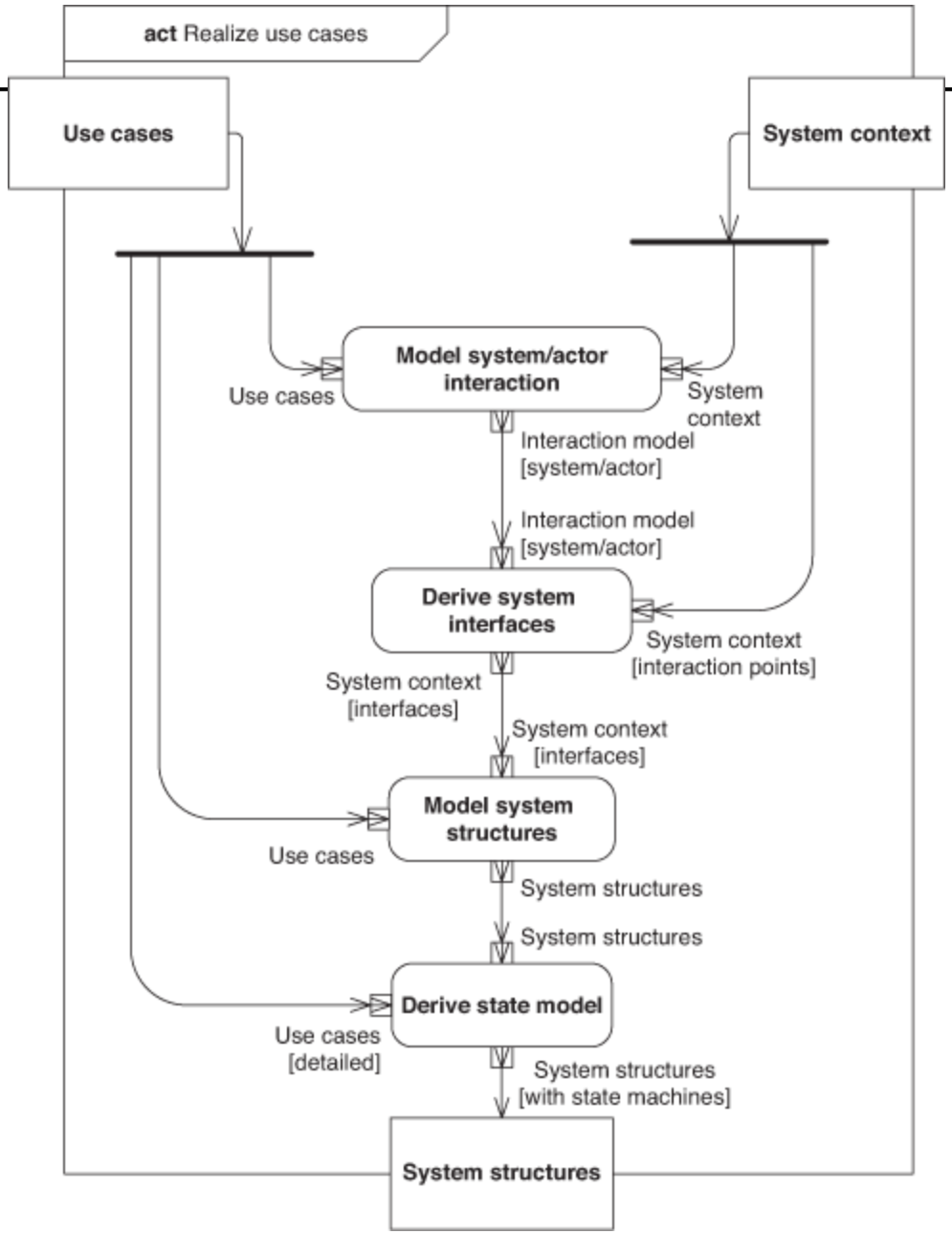
Question: what would it mean if an entire column has no allocations?

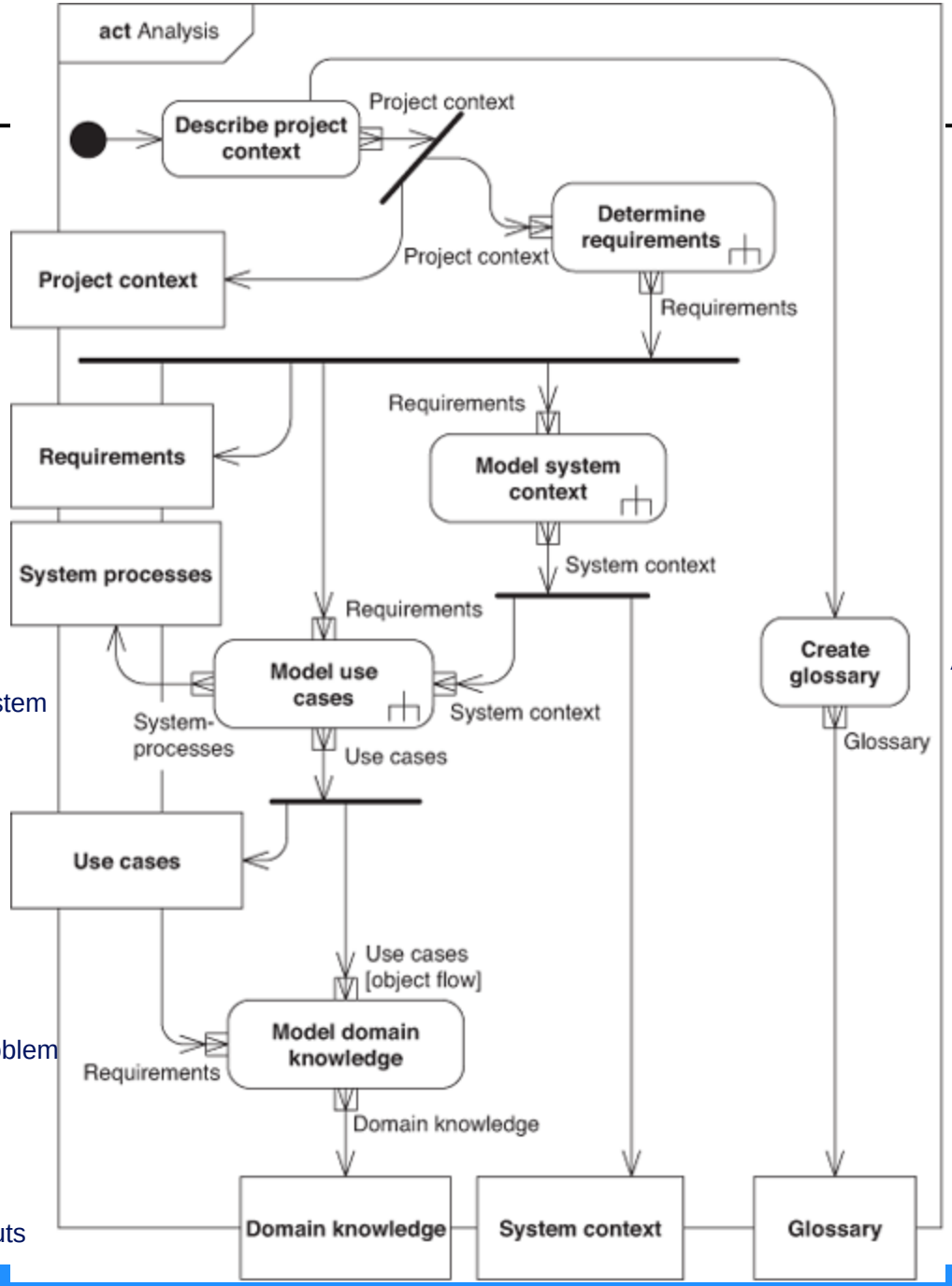
Question: Why use tables instead of diagrams?

- Automotive Sample Problem
  - Block specification and usage
  - …Including design part

- Commercial
  - **Rhapsody (IBM)** : used in aerospace, automotive, defense (Bosch, Airbus), dearw a state machine > it generates real C/C++ code runs on embedded systems, expensive
  - **Enterprise Architect**, more afordable than IBM, Supports SysML,UML, and many other notations in one tool.
- Free
  - Papyrus: Maintained by the Eclipse Foundation (SysMl, UML)
  - Powerful but takes patience to learn

- Details see: https://sysmltools.com/


act Realize use cases
Use cases
System context
Model system/actor interaction
Use cases
System context
Interaction model [system/actor]
Interaction model [system/actor]
Derive system interfaces
System context [interaction points]
System context [interfaces]
System context [interfaces]
Model system structures
Use cases
System structures
System structures
Derive state model
Use cases [detailed]
System structures [with state machines]
System structures



document

Ref: calls a sub-activity

All ways users interact with system

All terminology for the system

All expert knowledge about the problem

What domain experts know

What is inside/outside the system

All terminologies

Final Outputs

Discussion & Summary

- There exists different decomposition principles such as **functional structuring**, **data-flow structuring** and **object-oriented structuring** which may occur in combination when analyzing or designing a complex system.
- **SysML** (UML) provides a rich set of modeling concepts for the analysis and design of softwareintensive systems.
- While analysis focuses on the requirements and **problem domain**, the design describes the proposed **solution** (which might include some elements of the problem domain).

III.7 Bibliography

[Agha1986] G. A. Agha. ACTORS: A Model of Concurrent Computation in Distributed Systems. MIT Press, 1986.

[Booch1993] Grady Booch. Object-Oriented Analysis and Design with Applications. Addison-Wesley, Menlo Park CA, 1993. (Second Edition).

[Bruegge&Dutoit2003] Bernd Bruegge, Allen H. Dutoit. Object-Oriented Software Engineering: Using UML, Patterns and Java, Prentice Hall, Upper Saddle River, NJ, 2003. 2nd Edition.

[Buschmann+1996] F. Buschmann, R. Meunier, H. Rohnert, P. Sommerlad, and M. Stal. Pattern Oriented Software Architecture. John Wiley & Sons, Inc., 1996.

[Coleman+1994] Derek Coleman, Patrick Arnold, Stephanie Bodoff, Chris Dollin, Helena Gilchrist, Fiona Hayes, and Paul Jeremaes. Object-Oriented Development: The Fusion Method. Prentice-Hall, 1994.

[Cooling2002] Jim Cooling, Software Engineering for Real-time Systems. Addison Wesley, November 2002

[Douglass1998b] Bruce Powel Douglass. Safety Critical Systems. In Embedded Systems Conference, Spring 1998.

[Douglass1999] Bruce Powel Douglass. Doing Hard Time: Developing Real-Time Systems with UML, Objects, Frameworks and Patterns. Addison-Wesley, May 1999.

Bibliography

[Douglass1999b] Bruce Powel Douglass. Real-Time UML: Developing Efficient Objects for Embedded Systems. The Addison-Wesley Object Technology Series. Addison-Wesley, October 1999. Second Edition.

[IEEE-Std-610.12-1990] Standards Coordinating Committee of the IEEE Computer Society, USA. IEEE standard glossary of software engineering terminology, IEEE Std 610.12-1990, 1990.

[Gamma+1994] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. Design Patterns, Elements of Reusable Object-Oriented Software. Addison-Wesley, 1994.

[Heinecke2004] AUTOSAR: Harald Heinecke (BMW Group). Complexity management in automotive Industry. „nd VECTOR Congress 2004, 13. October 2004. (Slides)

[Korff2005] Andreas Korff (Artisan), AUTOSAR and UML 2.0 – a natural fit? Tutorial at the OMER3 Workshop, Paderborn, 12 October 2005. (Slides)

[Liskov&Zilles1974] Barbara H. Liskov and Stephen N. Zilles. Programming with Abstract Data Types. Techreport, 1974.

[Meyer1997] Bertrand Meyer. Object-Oriented Software Construction. Prentice Hall, 1997. 2nd edition.

Bibliography

[Meyer&Mingins1999] Bertrand Meyer and Christine Mingins. Component-based Development: From Buzz to Spark. IEEE Computer, 32(7):35–37, July 1999.

[Monarchi&Puhr1992] Monarchi, D. E. and Puhr, G. I. 1992. A research typology for object-oriented analysis and design. Commun. ACM 35, 9 (Sep. 1992), 35-47.

[Nagl1990] Manfred Nagl. Softwaretechnik: Methodisches Programmieren im Großen. Springer Verlag, 1990.

[Nerson1992] Nerson, J. 1992. Applying object-oriented analysis and design. Commun. ACM 35, 9 (Sep. 1992), 63-74.

[SysMLTutorial09]Sanford Friedenthal, Alan Moore, Rick Steiner, OMG Systems Modeling Language (OMG SysML™)Tutorial, September 2009

[Rumbaugh+1991] J. Rumbaugh, M. Blaha, W. Premerlani, F. Eddy, and W. Lorensen. Object-Oriented Modeling and Design. Prentice Hall, 1991.

[Seubert&Kämpfer2005] Tilmann Seubert and Elke Kämpfer. AUTOSAR: Intents, Status, Consequences. Keynote at the OMER 3 Workshop, Paderborn, 14 October 2005. (Slides)

[Shaw&Garlan1996]Mary Shaw and Davis Garlan. Software Architecture: Perspectives on an emerging Discipline. Prentice Hall, 1996.

[Shoham1992] Y. Shoham. Agent-oriented programming. Artifical Intelligence, 60:51–92, 1992.

[Szyperski1998] Clemens Szyperski. Component Software, Beyond Object-Oriented Programming. Addison-Wesley, 1998.

[Wirfs-Brock+1990] R. Wirfs-Brock, B. Wilkerson, and L. Wiener. Designing Object-Oriented Software. Prentice Hall, 1990.

[Wooldridge&Jennings1995] Michael Wooldridge and Nicholas R. Jennings. Agent Theories, Architectures, and Languages: a Survey. In Michael Wooldridge and Nicholas R. Jennings, editors, Intelligent Agents, pages 1–22. Springer Verlag, 1995.